Архитектура, алгоритмы и развертывание интерактивной веб-платформы для анализа поведенческого детерминизма

Философский контекст и теоретическое обоснование эксперимента

Проблема свободы воли и детерминизма представляет собой один из наиболее устойчивых и фундаментальных вопросов в истории философской мысли. На протяжении веков мыслители спорили о том, является ли человеческий выбор результатом независимого, спонтанного акта сознания, или же он жестко детерминирован предшествующими состояниями, скрытыми когнитивными паттернами и внешними стимулами. Классическая игра «Камень, ножницы, бумага» (КНБ), которая на первый взгляд кажется тривиальным генератором случайных исходов с нулевой суммой и совершенной информацией, в действительности выступает идеальным полигоном для эмпирического исследования поведенческого детерминизма.[1]

Фундаментальная проблема человеческого сознания заключается в его неспособности генерировать истинно случайные последовательности. Когда человеку предлагается действовать случайным образом, его мозг неизбежно опирается на эвристики, подсознательно формируя шаблоны, реагируя на предыдущие победы или поражения и пытаясь предвосхитить логику оппонента.[1] Интерактивный веб-эксперимент, в рамках которого человек соревнуется с алгоритмом машинного обучения (ML), способным в режиме реального времени выявлять и эксплуатировать эти скрытые психологические паттерны, служит мощным практическим доказательством концепции детерминизма. По мере того как машина учится предсказывать каждый следующий шаг пользователя со всё возрастающей точностью, философский аргумент о предсказуемости человеческого выбора получает неопровержимое математическое подтверждение.[3]

В контексте подготовки к научной конференции создание подобной интерактивной платформы требует глубокого понимания не только философской базы, но и современных технологий. Задача состоит в том, чтобы разработать систему, которая будет функционировать безупречно, адаптироваться под мобильные устройства участников в зале, выдерживать одновременную нагрузку без задержек и визуально транслировать идеи строгости и машинной логики. Для этого необходимо провести исчерпывающий анализ алгоритмических моделей, существующих проектов с открытым исходным кодом, актуального технологического стека 2025–2026 годов, а также стратегий развертывания

веб-приложений. Настоящий отчет представляет собой детальное исследование каждого из этих аспектов, предлагая оптимальный путь от концептуализации идеи до ее технической реализации и публичной демонстрации.

Эволюция алгоритмических моделей: От псевдослучайности к предиктивной аналитике

Для задачи предсказания ходов в КНБ применяются различные подходы, начиная от простейших генераторов случайных чисел и заканчивая сложными глубокими нейронными сетями. Выбор правильной математической модели является критическим фактором для успеха философского эксперимента, так как модель должна обучаться достаточно быстро, чтобы участник конференции смог осознать факт своей предсказуемости в течение короткой игровой сессии.

Базовые эвристические и скриптовые алгоритмы

Начальным уровнем понимания алгоритмического противостояния в КНБ является анализ простых ботов, использующих жесткие правила. В рамках образовательных проектов, таких как сертификация свободного кода по машинному обучению на Python, часто рассматриваются примитивные алгоритмы, с которыми должна соревноваться обучаемая модель.[4] Например, алгоритм может использовать повторяющийся паттерн определенной длины, такой как последовательность «Камень, бумага, бумага, ножницы, камень».[4] Более продвинутый скриптовый подход заключается в ведении словаря частотности, где алгоритм запоминает, какие ходы оппонент делал после определенной пары предыдущих ходов, и всегда выбирает вариант, бьющий наиболее вероятный следующий ход оппонента.[4]

Существуют также проекты, использующие генераторы псевдослучайных чисел (PRNG) с гауссовым распределением или регистры сдвига с линейной обратной связью (LFSR) для создания видимости сложного поведения.[5] В таких системах игра делится на стадии, и на каждой стадии алгоритм меняет доминирующий жест или дисперсию распределения, пытаясь запутать человека.[5] Однако все эти подходы являются реактивными или псевдослучайными; они не обладают способностью к истинному обучению и адаптации в реальном времени, что делает их непригодными для демонстрации глубокого поведенческого детерминизма.

Нейросетевые подходы: LSTM, CNN и обучение с подкреплением

Современные проекты на GitHub часто экспериментируют с глубоким обучением для решения задачи КНБ. Значительная часть таких проектов использует сверточные нейронные сети (CNN), созданные с помощью библиотек машинного зрения (например, TensorFlow Keras), для классификации изображений жестов рук через веб-камеру.[6] В этих реализациях сеть обучается на тысячах предварительно размеченных изображений,

распознавая пространственные паттерны для определения того, показывает ли пользователь камень, ножницы или бумагу.[6] Хотя это впечатляющая техническая задача, она решает проблему ввода данных, а не проблему предсказания будущего поведения.

Для прогнозирования временных рядов и последовательностей применяются рекуррентные нейронные сети, в частности сети долгой краткосрочной памяти (LSTM).[7] В архитектуре LSTM модель анализирует историю ходов как временной ряд, применяя ансамблевое принятие решений и динамическую скорость обучения.[7] Кроме того, существуют реализации на основе глубокого обучения с подкреплением (Deep Reinforcement Learning), где агент учится максимизировать награду путем взаимодействия со средой.[1]

Несмотря на математическую сложность, нейросетевые подходы обладают фатальным недостатком в контексте данного эксперимента — проблемой «холодного старта».[8] Глубокие нейронные сети требуют огромных объемов данных для предварительного обучения и значительного времени на корректировку синаптических весов в процессе игры. Участник конференции, играя на своем мобильном устройстве, сделает от 20 до 50 ходов. В таких условиях нейросеть просто не успеет сойтись к оптимальным значениям и выявить индивидуальные паттерны конкретного человека, в результате чего ее эффективность на коротких дистанциях не превысит эффективность случайного угадывания.[2]

Марковские цепи как оптимальное решение для интерактивной демонстрации

Анализ показывает, что идеальным решением для интерактивного веб-приложения, требующего обучения в реальном времени с первых же раундов, является использование цепей Маркова (Markov Chains).[9] Марковская модель опирается на фундаментальное марковское свойство: вероятность будущего состояния системы зависит исключительно от ее текущего состояния (или строго ограниченного числа предыдущих состояний), а не от всей истории целиком.[11] В контексте игры это означает, что следующий ход человека предсказывается на основе его предыдущего хода и, что критически важно, эмоционального результата предыдущего раунда.

Для реализации марковской модели используется матрица переходных вероятностей. Каждая ячейка матрицы $p_{i,j}$ представляет собой вероятность того, что пользователь выберет жест j при условии, что в предыдущем раунде (или серии раундов) он находился в состоянии i.[9]

Состояние i (предыдущий ход)	Вероятность выбрать Камень (j=R)	Вероятность выбрать Бумагу (j=P)	Вероятность выбрать Ножницы (j=S)
Камень (R)	p_{RR}	p_{RP}	p_{RS}
Бумага (P)	p_{PR}	p_{PP}	p_{PS}
Ножницы (S)	p_{SR}	p_{SP}	p_{SS}

Для вычисления вероятностей на лету алгоритм использует формулу, включающую поправку Лапласа (Laplace smoothing). Эта поправка предотвращает проблему «нулевой вероятности» на ранних этапах игры, когда пользователь еще не успел сделать все возможные комбинации ходов.[9] Формула оценки вероятности выглядит следующим образом [9]:

$$\hat{p}_{i,j} = \frac{N_{i \to j} + 1}{\sum_{k \in \{R,P,S\}} N_{i \to k} + V}$$

Где $V = 3$ (общее количество возможных состояний: Камень, Ножницы, Бумага), а N — количество фактически зафиксированных переходов пользователя из состояния i в состояние j.[9]

Многомерная оценка: Разделение матриц состояний

Для радикального повышения точности предсказаний базовая цепь Маркова модернизируется с учетом человеческой психологии. Люди ведут себя совершенно по-разному в ситуациях успеха, неудачи и неопределенности.[11] Модель, объединяющая все ходы в одну матрицу, теряет важный эмоциональный контекст. Поэтому передовой подход заключается в поддержке трех независимых таблиц переходов: WIN (Победа), LOSE (Поражение) и TIE (Ничья).[11] Если в предыдущем раунде человек выиграл с помощью Камня, а затем снова выбрал Камень, эта последовательность (Камень $\rightarrow$ Камень) записывается исключительно в таблицу WIN.[11] Алгоритм усваивает, склонен ли конкретный человек эксплуатировать победную стратегию, или же он предпочитает

менять тактику. При прогнозировании следующего хода машина сначала проверяет результат последнего раунда, выбирает соответствующую таблицу (например, LOSE), находит там строку с предыдущим ходом человека и выбирает тот следующий ход, который имеет максимальную вероятность.[11] После этого машина делает ход, который гарантированно бьет предсказанный выбор человека (если предсказаны Ножницы, машина ставит Камень).[11] Эта система является моделью активного обучения, которая непрерывно обновляется после каждого раунда.[11]

Глубина памяти и решение проблемы холодного старта

Человеческое поведение часто содержит паттерны, выходящие за рамки одного хода (например, чередование «Камень-Ножницы-Камень-Ножницы»). Для учета этого внедряются многоступенчатые марковские цепи переменной глубины, где состояние i определяется последовательностью из последних 2, 3 или 4 ходов.[9] Однако увеличение глубины усугубляет проблему нехватки данных в первых раундах. Для элегантного решения этой проблемы применяется система взвешенных прогнозов, использующая две матрицы: Глобальную (включающую агрегированную историю миллионов игр всех предыдущих людей) и Уникальную (содержащую историю только текущего пользователя).[9] На начальном этапе вес глобальной матрицы составляет 50%, обеспечивая базовое понимание человеческой психологии. По мере увеличения количества сыгранных раундов вес глобальной матрицы экспоненциально снижается до минимального порога в 25%, уступая место уникальной матрице, которая отражает специфические паттерны текущего игрока.[9] Это гарантирует, что машина будет играть сильно с самого первого хода.

Анализ существующих проектов: Использовать готовое или написать свое?

В экосистеме GitHub существует огромное количество реализаций игры в КНБ. Выбор между использованием готового решения (Buy/Fork) и разработкой собственного с нуля (Build) зависит от специфики академического проекта, требований к кастомизации и визуальной эстетике. Анализ репозиториев позволяет выделить три основные категории проектов.

Категория проектов	Типичные представители	Ключевые преимущества	Критические недостатки для философского проекта
Комплексные	depapp/rock-paper	Использование	Избыточная

современные решения	-scissors [3], adaapp/rock-paper-typescript.[14]	Next.js, TypeScript, Redis и интеграция с LLM (Google Gemini) для концепции Explainable AI, где алгоритм текстом объясняет, почему он предсказал тот или иной ход.[3] Высокая масштабируемость.	архитектурная сложность. Жесткая привязка к платным сторонним API (Gemini), что делает проект зависимым от квот.[3] Сложность локального развертывания на ноутбуке без настройки облачного Redis.
Готовые ML-скрипты без современного UI	iamvigneshwars/rock-paper-scissors-ai [11], milsaware/Javascript-AI-Rock-Paper-Scissors [15], uliseshdzc/ml-rock-paper-scissors.[10]	Идеально реализованная математическая логика цепей Маркова на JavaScript или Python. Полностью открытый код, малый вес алгоритма.[11] Разделение логики на WIN/LOSE/TIE таблицы.[11]	Устаревший, примитивный пользовательский интерфейс (часто консольный или базовая HTML-верстка).[16] Отсутствие адаптации под современные мобильные устройства. Несоответствие высоким эстетическим стандартам веб-дизайна 2025 года.
UI-ориентированные учебные проекты	Проекты из Frontend Mentor, Jefferson00/RockPaperScissors [18], rustysym/nextjs-rock-paper-scissors.[19]	Отличная современная верстка, использование React, Next.js, Framer Motion для плавных анимаций.[19]	Полное отсутствие машинного обучения. Логика компьютера сводится к Math.random(), что уничтожает всю философскую

			ценность эксперимента.

Рекомендация для научной работы: Оптимальным путем является гибридный подход. Взять существующий, сложный готовый проект (например, dеpapp) будет ошибкой из-за перегруженности инфраструктуры. Взять простой скрипт недопустимо из-за плохой графики. Идеальное решение — написать платформу самостоятельно, заимствуя математическое ядро (чистые функции цепей Маркова) из проектов вроде iamvigneshwars, и интегрируя их в современный архитектурный каркас на базе Next.js, который мы спроектируем с нуля. Это обеспечит абсолютный контроль над логикой, идеальную подгонку под мобильные устройства участников конференции и неповторимую эстетику, отражающую суть философского доклада.

Архитектура и технологический стек (Стандарты 2025–2026)

Для обеспечения максимальной производительности, мгновенного отклика и легкости разработки в 2025 году рекомендуется использовать унифицированный стек на базе JavaScript/TypeScript для всего приложения (Full-Stack JavaScript).[21] Эта парадигма позволяет использовать один язык программирования как для пользовательского интерфейса, так и для логики предсказания.

Проблема выбора серверного языка: Node.js против Python

Существует распространенное заблуждение, что любые проекты, связанные с машинным обучением, должны использовать Python на бэкенде из-за наличия таких библиотек, как TensorFlow или PyTorch.[22] Однако детальный анализ показывает, что для данного специфического приложения Node.js является технически превосходящим выбором.

Python демонстрирует блестящие результаты в задачах, требующих интенсивных вычислений на процессоре (CPU-bound tasks), и обладает гигантской экосистемой для работы с массивами данных (NumPy, Pandas).[22] Однако Python традиционно страдает от ограничений глобальной блокировки интерпретатора (GIL), что усложняет параллельную обработку тысяч одновременных сетевых подключений без сложной настройки асинхронных фреймворков или мультипроцессинга.[22]

Node.js, напротив, построен на архитектуре однопоточного неблокирующего цикла событий (Event Loop).[22] Эта архитектура позволяет серверу тратить ровно 0% процессорного времени на ожидание завершения операций ввода-вывода (I/O). Когда пользователь делает ход, Node.js принимает запрос, инициирует обновление данных в оперативной памяти и мгновенно переходит к обработке запроса следующего пользователя.[24] Реальные бенчмарки показывают, что Node.js работает на 40–70%

быстрее в задачах с высокой интенсивностью ввода-вывода и обеспечивает значительно более низкую задержку (latency) при обработке множественных одновременных подключений по сравнению с классическими реализациями на Python.[22]

Более того, математические операции для цепей Маркова (сложение частот, деление, выбор максимального элемента из массива) настолько тривиальны, что JIT-компилятор движка V8 в Node.js выполняет их с колоссальной скоростью.[22] Даже если бы использовались нейронные сети, библиотека TensorFlow.js Node использует те же самые оптимизированные C++ библиотеки под капотом, что и Python, но за счет JIT-компиляции самого JavaScript время сквозной обработки запросов может быть потенциально в два раза быстрее.[26] Таким образом, использование единой кодовой базы на TypeScript является самым эффективным архитектурным решением.

Фронтенд: Фреймворк Next.js и React Server Components

Фундаментом платформы должен стать фреймворк Next.js, использующий инновационную парадигму App Router, которая стала стандартом маршрутизации в 2025 году.[27] В отличие от классических одностраничных приложений (SPA), сильно нагружающих процессор мобильного устройства, Next.js по умолчанию использует React Server Components (RSC).[28] Эта архитектура позволяет отрендерить значительную часть пользовательского интерфейса на сервере, передавая на мобильные устройства участников конференции только легковесный, оптимизированный HTML-код.[28] За счет автоматического разделения кода (Code Splitting) каждый компонент загружает только необходимый JavaScript, что радикально сокращает время начальной загрузки и обеспечивает идеальный пользовательский опыт даже при медленном мобильном интернете в переполненном конференц-зале.[29]

Стилизация: Tailwind CSS v4

Для создания сложного, адаптивного дизайна в кратчайшие сроки используется Tailwind CSS четвертой версии.[28] Tailwind представляет собой утилитарный фреймворк, который позволяет применять стили непосредственно через классы в разметке компонентов. Ключевым преимуществом Tailwind CSS в связке с серверными компонентами Next.js является полное отсутствие затрат времени на выполнение (No Runtime Cost).[28] В отличие от CSS-in-JS библиотек, JIT-компилятор Tailwind анализирует код во время сборки проекта (build time) и генерирует минимально возможный статический CSS-файл, содержащий только те классы, которые реально используются в проекте.[28] Это гарантирует абсолютную предсказуемость стилей и молниеносную отрисовку страницы на мобильных экранах. Встроенные инструменты Tailwind для создания адаптивного дизайна (Media Queries & Responsive Design) позволяют реализовать концепцию Mobile-First без написания сложных кастомных медиа-запросов.[29]

Микроинтеракции и анимация: Framer Motion

Философский эксперимент требует определенной драматургии: машина не должна выдавать ответ мгновенно, чтобы не создавать ощущение простого скрипта. Ответ должен появляться с небольшой, выверенной задержкой, сопровождаясь плавной визуализацией процесса «мышления» алгоритма. Для программирования комплексных анимаций в экосистеме React безусловным лидером является библиотека Framer Motion.[31] Вместо сложного управления CSS-переходами, Framer Motion использует декларативный подход. Компоненты оборачиваются в <motion.div>, где разработчик просто описывает начальное состояние (initial={{ opacity: 0, y: 30 }}) и целевое состояние (animate={{ opacity: 1, y: 0 }}).[31] Библиотека берет на себя все математические расчеты физики пружин, создавая естественные, плавные движения.[31] Использование компонента <AnimatePresence> позволяет бесшовно анимировать появление и удаление элементов из DOM дерева (например, смену экранов от этапа выбора к этапу результатов).[31] Комбинация Tailwind CSS для геометрии и типографики и Framer Motion для кинетики формирует самый актуальный стек создания интерфейсов 2025 года.[31]

Философская эстетика UI/UX: Нео-брутализм как отражение детерминизма

Дизайн платформы, сопровождающей глубокую философскую работу, не может быть просто "красивым" в традиционном, корпоративном смысле. Он должен транслировать смыслы. Классический, перегруженный градиентами, мягкими тенями и излишним декором интерфейс отвлекает субъекта от сути происходящего. Наиболее релевантной и мощной парадигмой веб-дизайна для демонстрации концепций детерминизма и неумолимой алгоритмической логики является Нео-брутализм (Neo-Brutalism).[33]

Исторические корни и перенос в цифровое пространство

Брутализм возник в середине XX века как архитектурное течение, основанное архитекторами Элисон и Питером Смитсонами в послевоенной Европе.[35] Главной идеологией движения была «честность материалов» (от французского *béton brut* — сырой, необработанный бетон), отказ от украшательства и демонстрация сырой функциональности и структурной обнаженности зданий.[35] Это был архитектурный ответ, провозглашающий прямолинейность в противовес визуальному комфорту.[35]

При переходе в цифровое пространство, веб-брутализм сохранил эту философию, отвергая лоск современных пользовательских интерфейсов. В вебе этот стиль часто принимал форму подчеркнуто утилитарных, "анти-дизайнерских" сайтов с сырым HTML, отсутствием стилизации и намеренной жесткостью.[33] Для философского проекта о предсказуемости человека это идеальная метафора: алгоритм срывает маски с иллюзии свободы воли, демонстрируя «сырую», машинную, детерминированную природу

человеческого выбора без каких-либо смягчающих декораций.

Формирование эстетики Нео-брутализма

Однако чистый брутализм может быть слишком радикален и вызывать отторжение у пользователей из-за проблем с юзабилити.[34] В 2025 году на смену ему пришел Нео-брутализм — современный подход, который сохраняет идеологию функциональности поверх формы, но объединяет ее с принципами минимализма, добавляя высокую контрастность и четкую навигацию.[34]

Визуальный язык Нео-брутализма для данного проекта должен строиться на следующих паттернах:

1. **Сверхкрупная типографика:** Текст становится главным архитектурным элементом.[33] Моноширинные (monospace) или жирные гротескные шрифты должны использоваться для отображения счета игры (Поражения человека / Победы машины). Цифры должны доминировать на экране, подчеркивая неумолимость статистики.
2. **Жесткая цветовая палитра:** Рекомендуется монохромный дизайн (абсолютно черный фон, чистый белый текст и жесткие серые разделители) с внедрением строго одного или двух высоконасыщенных акцентных цветов.[33] Например, использование агрессивного красного или неоново-желтого цвета исключительно в моменты, когда алгоритм выигрывает раунд. Никаких градиентов или полутонов.
3. **Визуальные формы и тени:** Отказ от сглаженных углов (border-radius: 0). Кнопки выбора жестов (Камень, Ножницы, Бумага) должны быть подчеркнуто геометричными, иметь толстые черные контуры (outlines) и жесткие, сплошные падающие тени без размытия, создающие иллюзию плоского, но объемного пространства.[34]
4. **Кинетика:** Анимации, реализуемые через Framer Motion, должны быть резкими и мгновенными.[33] Вместо эластичных отскоков элементы должны появляться с математической точностью, отражая холодный, выверенный расчет машины.

Принципы проектирования под мобильные устройства (Mobile-First)

Учитывая, что в условиях научной конференции аудитория будет взаимодействовать с платформой через свои смартфоны, разработка должна начинаться исключительно с мобильной версии (wireframes).[38] Интерфейс должен помещаться на одном экране без необходимости вертикального скроллинга.[38] Верхняя часть экрана отводится под информационное табло (Текущий счет и процент уверенности нейросети в следующем ходе). Центральная часть отображает арену (выбор человека против выбора машины). Нижняя часть, находящаяся в зоне комфортной досягаемости большого пальца пользователя (Thumb Zone), должна содержать массивные, контрастные кнопки ввода жестов. Отказ от лишних окон подтверждения (например, необходимости нажимать кнопку "OK" после каждого раунда) радикально улучшит динамику игры и позволит

быстрее собирать данные для алгоритма.[39]

Инфраструктура, масштабирование и стратегии развертывания

Для успешной демонстрации проекта на научной конференции критически важно обеспечить доступность сайта для целевой аудитории (ориентировочно 30 человек), гарантировать отсутствие задержек, и предоставить простой способ подключения (например, вывод QR-кода со ссылкой на экран проектора). При этом проект должен минимизировать финансовые затраты. Существует два принципиально разных архитектурных подхода: классическое облачное развертывание и локальное туннелирование с устройства докладчика.

Управление нагрузкой: Парадокс 30 пользователей

Прежде чем выбирать платформу, необходимо развеять мифы о масштабировании. 30 одновременных пользователей, играющих в браузерную игру на базе текстовых JSON-обменов, генерируют ничтожно малую нагрузку на современные вычислительные системы. Как упоминалось ранее, архитектура Node.js, используемая в основе Next.js, оперирует единым циклом событий (Event Loop).[22] В отличие от многопоточных серверов, где каждое новое соединение от пользователя требует выделения отдельного потока в операционной системе со своей памятью (что быстро истощает ресурсы), Node.js асинхронно ждет поступления данных в один поток. Получив сигнал о ходе игрока, процессор за доли миллисекунды обновляет марковскую матрицу в памяти, вычисляет ответ, отправляет его обратно и переходит в состояние ожидания следующего пакета данных.[24] В результате, даже простейший инстанс сервера или обычный ноутбук способен с легкостью поддерживать десятки тысяч одновременных TCP/WebSocket соединений, используя при этом минимальный объем оперативной памяти и процессорного времени.[25] Основной проблемой является не нагрузка на процессор (CPU-bound), а сетевая задержка (latency) между устройством пользователя и дата-центром.[40]

Вариант 1: Полное облачное развертывание (Cloud Hosting)

Использование облачных платформ (PaaS) является индустриальным стандартом для долговременных проектов. Экосистема предлагает мощные бесплатные уровни (Free Tiers), однако они имеют свои ограничения.

Платформа	Роль в проекте	Лимиты бесплатного тарифа (2025-2026)	Анализ применимости

Vercel	Хостинг фронтенда (Next.js)	Тариф Hobby: 1 миллион edge-запросов в месяц, до 100 деплоев в день. Максимальное время выполнения бессерверной функции (Serverless Function) — 60 секунд.[42]	Идеальное решение для размещения статики и легкого API. Vercel создан разработчиками Next.js, что гарантирует идеальную оптимизацию кэширования (CDN) и мгновенную отдачу HTML-кода пользователям во всем мире.[45]
Render	Хостинг постоянного бэкенда (если требуется WebSocket или хранение состояния в памяти между сессиями)	100 одновременных соединений для веб-сервисов, 750 часов работы в месяц.[47]	Ограничение в 100 соединений с запасом покрывает аудиторию в 30 человек.[47] Однако главный недостаток бесплатного тарифа Render — "холодный старт" (Cold Start). Если к серверу нет обращений в течение 15 минут, он засыпает, и первый запрос от пользователя может занимать до 50 секунд на пробуждение контейнера, что недопустимо во время живой презентации.[45]

Облачный подход имеет смысл, если планируется оставить проект доступным для широкой публики после конференции. Однако для локального мероприятия с контролируемым запуском существует более изящное решение.

Вариант 2: Локальное развертывание с пробросом туннеля (Conference Demo Mode)

Оптимальной стратегией для научной конференции является развертывание всей логики и базы данных непосредственно на ноутбуке выступающего. Это решает массу проблем: код запускается мгновенно, не страшны лимиты облачных провайдеров, а любые сетевые задержки сводятся к скорости Wi-Fi в конференц-зале. Единственная задача — безопасно выставить локальный сервер (например, работающий на адресе localhost:3000) в публичный интернет для получения публичной ссылки.

В прошлом для этих целей массово использовался инструмент **ngrok**.[50] Однако в 2025 году условия его бесплатного тарифа делают его непригодным для публичных демонстраций.[51] Бесплатный тариф ngrok накладывает жесткое ограничение на 1 ГБ трафика в месяц и, что более критично, принудительно разрывает сессии каждые 2 часа, требуя ручного перезапуска туннеля и смены URL-адреса.[51] Кроме того, установлен лимит в 5000 TCP-соединений в месяц, который 30 пользователей, играющих десятки раундов, могут исчерпать за время выступления.[52]

Технологический стандарт 2025 года: Cloudflare Tunnel (бывший Argo Tunnel)
Инструмент от Cloudflare лишен недостатков ngrok. Он позволяет абсолютно бесплатно, безопасно и без ограничений по трафику или времени сессии экспонировать локальный порт ноутбука в интернет.[54] Туннель работает за счет создания зашифрованного исходящего соединения от вашего компьютера к ближайшему дата-центру Cloudflare, обходя необходимость сложной настройки переадресации портов (port forwarding) на маршрутизаторе провайдера интернета.[54] Запросы пользователей попадают на высокоскоростные серверы Cloudflare и по туннелю передаются на ноутбук лектора.[56]

Пошаговый протокол настройки Cloudflare Tunnel для презентации [54]:

1. **Подготовка домена:** Необходимо приобрести или использовать любой имеющийся домен (например, philosophy-experiment.com). Домен добавляется в панель управления Cloudflare (выбирается тарифный план Free). В настройках регистратора домена прописываются DNS-серверы, выданные Cloudflare (например, carter.ns.cloudflare.com).[54]
2. **Создание туннеля:** В консоли управления Cloudflare необходимо перейти в раздел Zero Trust $\rightarrow$ Networks $\rightarrow$ Tunnels. Нажать "Create a tunnel" и присвоить ему техническое имя.[54]
3. **Установка демона:** На ноутбук докладчика скачивается и устанавливается легковесный клиент cloudflared (с помощью Homebrew для macOS, пакетного

менеджера для Linux или бинарного файла для Windows).[54]

4. **Связывание устройства с облаком:** В терминале ноутбука выполняется команда, сгенерированная в панели Cloudflare (вида cloudflared service install <УНИКАЛЬНЫЙ_ТОКЕН>), авторизующая туннель.[54]
5. **Маршрутизация (Public Hostname):** В настройках созданного туннеля во вкладке Public Hostname добавляется запись. Указывается публичный домен, протокол HTTP и целевой локальный адрес (например, localhost:3000, где запущен Next.js сервер).[54]
6. **Безопасность и SSL:** Cloudflare автоматически выпускает и поддерживает SSL-сертификат, обеспечивая защищенное HTTPS-соединение.[54] IP-адрес ноутбука остается полностью скрытым от интернета, что гарантирует защиту от DDoS-атак.[55]

Сгенерированную красивую ссылку докладчик конвертирует в QR-код и выводит на последний слайд презентации. Слушатели сканируют код и мгновенно подключаются к локальному серверу лектора.

Индустриальные стандарты и лучшие практики разработки платформы

Для того чтобы платформа функционировала безупречно и вызывала доверие в научной среде, архитектура должна быть спроектирована с учетом строгих инженерных стандартов:

1. Архитектура управления состоянием (State Management)

Данные об истории ходов разных пользователей не должны смешиваться, иначе модель цепей Маркова потеряет смысл, усредняя паттерны поведения разных людей.[3] Как при локальном, так и при облачном развертывании, каждому пользователю при открытии страницы должен генерироваться уникальный идентификатор сессии (UUID), который сохраняется в localStorage его браузера или в HTTP-cookie.[3] На сервере создается изолированное пространство памяти (объект в JavaScript или запись в Redis), где хранится персональная трехмерная матрица переходов (WIN/LOSE/TIE) для этого конкретного пользователя. В начале игры эта матрица обогащается глобальными весами, но затем эволюционирует независимо.[3]

2. Информационная безопасность и защита от манипуляций

Важнейшее правило разработки интерактивных игр: **клиент никогда не должен управлять логикой**.[54] Весь код цепей Маркова, расчет вероятностей и принятие решения о следующем ходе машины должны физически находиться на сервере (внутри API Routes фреймворка Next.js). Клиентский код (браузер мобильного устройства) должен отвечать исключительно за отрисовку красивого UI и отправку простых JSON-объектов с выбранным жестом на сервер (например, POST /api/move { gesture: "ROCK" }). Сервер принимает данные, обновляет матрицу, вычисляет предсказание для следующего раунда,

скрыто фиксирует его и возвращает клиенту результат текущего раунда. Если алгоритм будет выполняться в браузере, любой технически подкованный слушатель сможет открыть панель инструментов разработчика (DevTools), извлечь матрицу вероятностей, прочитать следующий ход машины и начать выигрывать 100% раз, дискредитировав философский эксперимент.

3. Интеграция концепции Объяснимого ИИ (Explainable AI)

Поскольку фундаментальная цель проекта — академическая демонстрация, недостаточно просто создать машину, которая обыгрывает человека. Машина должна демонстрировать свою логику, чтобы усилить философский шок от осознания собственной предсказуемости. Вдохновляясь передовыми проектами, необходимо внедрить элементы "прозрачного алгоритма".[3] На интерфейсе каждого пользователя, в режиме реального времени, должны выводиться вероятностные распределения. Например, перед тем как человек сделает ход, машина может выводить сообщение: *"Основываясь на вашем предыдущем поражении, вероятность того, что вы выберете Ножницы составляет 72%, Камень — 19%, Бумагу — 9%"*. Визуализация того, как машина поглощает поведенческие шаблоны, превращает абстрактную концепцию детерминизма в пугающую, наглядную реальность для каждого участника.[3]

4. Телеметрия, агрегация данных и аналитика

Серверная часть должна вести строгий, анонимный лог всех транзакций. Каждый ход каждого из 30 участников, с привязкой к временным меткам (Time-Series Data) и флагами успешности предсказаний алгоритма, должен записываться в единую структуру данных (лог-файл или легковесную базу данных SQLite).[3] Наличие этого массива данных является критически важным инструментом для завершающей части доклада на конференции. Спустя 10-15 минут интерактива, лектор должен вывести на главный экран общую панель управления (Dashboard), где будут показаны агрегированные результаты. Например: *"За последние 15 минут вы коллективно сыграли 1200 раундов. Начиная с 15-го раунда каждого игрока, точность алгоритма выросла с базовых 33.3% до стабильных 78%"*. Эти твердые статистические данные послужат неопровержимым эмпирическим доказательством гипотезы о том, что свобода воли в подобных сценариях иллюзорна, а поведение жестко детерминировано предыдущим опытом и подлежит точному математическому моделированию.

Синтез

Создание интерактивной веб-платформы для противостояния человека и машины в игру с неполной информацией предоставляет уникальный инструментарий для исследования и демонстрации философских концепций детерминизма. Успех проекта зависит от синергии математической точности, передовых веб-технологий и глубоко продуманного дизайна.

Глубокий анализ показывает, что для задачи быстрого адаптивного обучения в рамках коротких сессий наиболее мощным и элегантным решением является применение марковских цепей с динамической глубиной и использованием независимых матриц переходов (WIN, LOSE, TIE), которые учитывают психоэмоциональное состояние человека.[11] В отличие от тяжеловесных нейросетей (LSTM, CNN), марковская модель не страдает от проблемы длительного холодного старта, позволяя алгоритму начать эффективно противодействовать паттернам пользователя уже после нескольких раундов.[2]

С архитектурной точки зрения, попытки адаптации существующих сложных проектов (нагруженных сторонними LLM или громоздкими базами данных) приведут к избыточной сложности развертывания.[3] Наилучшим решением является собственная разработка на едином стеке TypeScript, где математическое ядро Маркова интегрируется в инновационный каркас Next.js с серверными компонентами.[28] Выбор Node.js для серверных вычислений обеспечивает идеальную обработку множества одновременных подключений благодаря событийной неблокирующей архитектуре, превосходя традиционные синхронные подходы Python в задачах I/O реального времени.[22]

Визуальное оформление должно строго следовать эстетике Нео-брутализма, отказываясь от декоративности в пользу сырой функциональности, гигантской типографики и жестких геометрических форм, которые метафорически отражают безжалостность алгоритмической логики.[33] Интеграция микроинтеракций через Framer Motion добавит необходимую кинетическую выверенность каждому движению интерфейса.[31]

Наконец, для живой демонстрации перед аудиторией оптимальным решением выступает локальное развертывание платформы на вычислительных мощностях докладчика с созданием защищенного туннеля через Cloudflare Tunnel. Это полностью устраняет зависимость от жестких ограничений бесплатных облачных провайдеров (таких как лимиты сессий в ngrok или засыпание серверов Render), гарантирует нулевую стоимость инфраструктуры и обеспечивает безупречный, мгновенный доступ для всех участников конференции через их мобильные устройства.[48] Реализованная таким образом система станет не просто техническим фокусом, а строгим, эмпирически обоснованным аргументом в философской дискуссии о природе человеческого сознания.

Works cited

1. Pattern Recognition and Reinforcement Learning — US Pycon ..., accessed March 31, 2026, https://rhettinger.github.io/rock_paper.html
2. How to win over 70% matches in Rock Paper Scissors | by Piotr Gabrys - Medium, accessed March 31, 2026, https://medium.com/data-science/how-to-win-over-70-matches-in-rock-paper-scissors-3e17e67e0dab
3. depapp/rock-paper-scissors: an AI Prediction Game · GitHub - GitHub, accessed

March 31, 2026, https://github.com/depapp/rock-paper-scissors
4. Rock Paper Scissors in Python - by Sri Hartini - Medium, accessed March 31, 2026, https://medium.com/@sri.hartini/rock-paper-scissors-in-python-5173ab69ca7a
5. dennylslee/rock-paper-scissors-DeepRL: rock paper scissors game using Double-DQN against different random generator - GitHub, accessed March 31, 2026, https://github.com/dennylslee/rock-paper-scissors-DeepRL
6. Rock, paper or scissors convolutional neural network - GitHub, accessed March 31, 2026, https://github.com/dieegogutierrez/rock-paper-scissors-model
7. Rock Paper Scissors AI - GitHub, accessed March 31, 2026, https://github.com/dmickelson/Rock-paper-scissors
8. ML with Python - Rock Paper Scissors - The freeCodeCamp Forum, accessed March 31, 2026, https://forum.freecodecamp.org/t/ml-with-python-rock-paper-scissors/461657
9. Creating a Machine Learning Cloud stack to play rock paper scissors -, accessed March 31, 2026, https://alexbaron.es/mlops/rps_bot/
10. An implementation of Markov chains to play rock-paper-scissors - GitHub, accessed March 31, 2026, https://github.com/uliseshdzc/ml-rock-paper-scissors
11. iamvigneshwars/rock-paper-scissors-ai: The AI based on ... - GitHub, accessed March 31, 2026, https://github.com/iamvigneshwars/rock-paper-scissors-ai
12. Rock Paper Scissors Simulation with Markov Chain Keeping Score - Stack Overflow, accessed March 31, 2026, https://stackoverflow.com/questions/55562239/rock-paper-scissors-simulation-with-markov-chain-keeping-score
13. I built a Rock–Paper–Scissors AI using a Markov model. It ties random players and exploits biased ones. - Reddit, accessed March 31, 2026, https://www.reddit.com/r/learnmachinelearning/comments/1qvr36q/i_built_a_rockpaperscissors_ai_using_a_markov/
14. Build the classic rock, paper, scissors game using TypeScript - GitHub, accessed March 31, 2026, https://github.com/adaapp/rock-paper-typescript
15. milsaware/Javascript-AI-Rock-Paper-Scissors: A rock ... - GitHub, accessed March 31, 2026, https://github.com/milsaware/Javascript-AI-Rock-Paper-Scissors
16. chibui191/rock-paper-scissors: The Odin Project - Web 101 Project - GitHub, accessed March 31, 2026, https://github.com/chibui191/rock-paper-scissors
17. codamee/rock-paper-scissors - GitHub, accessed March 31, 2026, https://github.com/codamee/rock-paper-scissors
18. Jefferson00/RockPaperScissors: A Next.js app of rock, paper, scissors game. - GitHub, accessed March 31, 2026, https://github.com/Jefferson00/RockPaperScissors
19. rustysym/nextjs-rock-paper-scissors - GitHub, accessed March 31, 2026, https://github.com/rustysym/nextjs-rock-paper-scissors
20. Rock-Paper-Scissors(reactJs-Framer motion) coding challenge solution - Frontend Mentor, accessed March 31, 2026, https://www.frontendmentor.io/solutions/rockpaperscissorsreactjsframer-motion-KxQ-wvYme5
21. The Best Tech Stacks for AI-Powered Applications in 2025 - DEV Community,

accessed March 31, 2026, https://dev.to/elliot_brenya/the-best-tech-stacks-for-ai-powered-applications-in-2025-efe
22. Node.js vs Python: Real Benchmarks, Performance Insights, and Scalability Analysis, accessed March 31, 2026, https://dev.to/m-a-h-b-u-b/nodejs-vs-python-real-benchmarks-performance-insights-and-scalability-analysis-4dm5
23. Python vs JavaScript for Backend: Which should you Learn in 2025? - Blog, accessed March 31, 2026, https://blog.masteringbackend.com/python-vs-java-script-for-backend-which-should-you-learn-in-2025
24. How, in general, does Node.js handle 10,000 concurrent requests? - Stack Overflow, accessed March 31, 2026, https://stackoverflow.com/questions/34855352/how-in-general-does-node-js-handle-10-000-concurrent-requests
25. How many concurrent users can Node.js support with socket.io framework? - Reddit, accessed March 31, 2026, https://www.reddit.com/r/node/comments/4x71a6/how_many_concurrent_users_can_nodejs_support_with/
26. Discuss pros/cons between Tensorflow Core and Tensorflow Js please, accessed March 31, 2026, https://discuss.ai.google.dev/t/discuss-pros-cons-between-tensorflow-core-and-tensorflow-js-please/19016
27. The Most Cutting-Edge Technologies and Website Frameworks for High-End Web Development in 2025 - ATAK Interactive, accessed March 31, 2026, https://www.atakinteractive.com/blog/the-most-cutting-edge-technologies-and-website-frameworks-for-high-end-web-development-in-2025
28. Next.js and Tailwind CSS 2025 Guide: Setup, Tips, and Best Practices - DEV Community, accessed March 31, 2026, https://dev.to/codeparrot/nextjs-and-tailwind-css-2025-guide-setup-tips-and-best-practices-2f6h
29. Crafting Responsive and Interactive UIs with Next.js and Tailwind CSS in 2025 | by @rnab, accessed March 31, 2026, https://arnab-k.medium.com/crafting-responsive-and-interactive-uis-with-next-js-and-tailwind-css-in-2025-6e72787fa788
30. Tailwind CSS v4 Full Course 2025 | Master Tailwind in One Hour - YouTube, accessed March 31, 2026, https://www.youtube.com/watch?v=6biMWgD6_JY
31. Framer Motion + Tailwind: The 2025 Animation Stack - DEV Community, accessed March 31, 2026, https://dev.to/manukumar07/framer-motion-tailwind-the-2025-animation-stack-1801
32. Responsive Animated Website with Tailwind CSS & Framer Motion Step by Step Tutorial for Freshers - YouTube, accessed March 31, 2026, https://www.youtube.com/watch?v=vE8dlK34e2o
33. Brutalism in Web Design: A Comprehensive Guide with Examples - TodayMade,

accessed March 31, 2026, https://www.todaymade.com/blog/brutalist-web-design
34. The Rise of Brutalism in Web Design: A Comprehensive Guide - DesignMantic, accessed March 31, 2026, https://www.designmantic.com/blog/brutalism-in-web-design/
35. What is Brutalism? A Guide to Brutalist Design, Architecture & Web Design - Cider House Media, accessed March 31, 2026, https://ciderhouse.media/brutalism-a-guide-to-architecture-web-design/
36. Redefining Web Brutalism - The Interconnected, accessed March 31, 2026, https://theinterconnected.net/ragdoll/redefining-web-brutalism/
37. The Split Personality Of Brutalist Web Development - Smashing Magazine, accessed March 31, 2026, https://www.smashingmagazine.com/2020/01/split-personality-brutalist-web-development/
38. Paper, Rock, Scissors Game Planning Guide - DEV Community, accessed March 31, 2026, https://dev.to/jcsmileyjr/paper-rock-scissors-game-planning-guide-40ad
39. Rock-paper-scissors with UI (The Odin Project) : r/learnjavascript - Reddit, accessed March 31, 2026, https://www.reddit.com/r/learnjavascript/comments/111hf9r/rockpaperscissors_with_ui_the_odin_project/
40. NestJS, Node.js, latency lies, damn lies, and statistics. | by Alex Turner - Medium, accessed March 31, 2026, https://medium.com/@alexrmturner/nestjs-node-js-latency-lies-damn-lies-and-statistics-fa2e93573d27
41. Latency issues with Node JS game and VPS - HTML5 Game Devs Forum, accessed March 31, 2026, https://www.html5gamedevs.com/topic/31399-latency-issues-with-node-js-game-and-vps/
42. Limits - Vercel, accessed March 31, 2026, https://vercel.com/docs/limits
43. Breaking down Vercel's 2025 pricing plans quotas and hidden costs | Flexprice, accessed March 31, 2026, https://flexprice.io/blog/vercel-pricing-breakdown
44. Vercel AI Pricing Plans 2026: How Much Does It Cost? - TrueFoundry, accessed March 31, 2026, https://www.truefoundry.com/blog/understanding-vercel-ai-gateway-pricing
45. Best Hosting Platform in 2025: My Hands On Guide to Choosing the Right One, accessed March 31, 2026, https://levelup.gitconnected.com/best-hosting-platform-in-2025-my-hands-on-guide-to-choosing-the-right-one-15e43b8dfee4
46. 15 Best React Hosting Providers in 2025 - Angular Minds, accessed March 31, 2026, https://www.angularminds.com/blog/react-hosting-providers
47. Pricing | Render, accessed March 31, 2026, https://render.com/pricing
48. Confused about the free tier - Render Community Migration, accessed March 31, 2026, https://community.render.com/t/confused-about-the-free-tier/19092
49. Connection Limits? - Render Community Migration, accessed March 31, 2026,

https://community.render.com/t/connection-limits/801
50. Usage Charges and Metrics in ngrok Plans, accessed March 31, 2026, https://ngrok.com/docs/pricing-limits/how-ngrok-charges
51. Ngrok and the Legacy Problem: Why Modern Developers Need Better Alternatives in 2025 | by InstaTunnel | Medium, accessed March 31, 2026, https://medium.com/@instatunnel/wngrok-and-the-legacy-problem-why-modern-developers-need-better-alternatives-in-2025-1b9912e62187
52. Free Plan Limits - ngrok documentation, accessed March 31, 2026, https://ngrok.com/docs/pricing-limits/free-plan-limits
53. ngrok Pricing — API Gateway, Tunnels & Kubernetes Ingress, accessed March 31, 2026, https://ngrok.com/pricing
54. How to Expose Your Local Server to the Internet (Without Port ..., accessed March 31, 2026, https://dev.to/mrviduus/how-to-expose-your-local-server-to-the-internet-without-port-forwarding-3f3h
55. Host Localhost Online for FREE! (No Port Forwarding) - YouTube, accessed March 31, 2026, https://www.youtube.com/shorts/5m1uFsQVzyc
56. How do you expose your self-hosted server to the internet? : r/selfhosted - Reddit, accessed March 31, 2026, https://www.reddit.com/r/selfhosted/comments/1fxg2m7/how_do_you_expose_your_selfhosted_server_to_the/